

Implementazione del Gioco UNO

Francesco Bartolucci – Matricola 2274957

Anno Accademico 2025/2026

1 Descrizione generale del progetto

1.1 Introduzione

Questo progetto consiste nello sviluppo del gioco di carte **UNO**, realizzato in **Java SE 25**. L'applicazione offre un'interfaccia grafica interattiva sviluppata con le librerie **Java Swing** e **AWT**.

Il programma implementa le principali meccaniche del gioco UNO, insieme ad alcune semplificazioni e varianti descritte nelle sezioni successive:

- Un mazzo di **108 carte**, suddivise tra carte numeriche (da 0 a 9 nei quattro colori Rosso, Giallo, Verde e Blu), carte azione speciali (*Skip*, *Reverse*, *Draw Two*) e carte jolly (*Wild* e *Wild Draw Four*);
- Supporto per partite da 2 a 6 giocatori, con gestione dinamica dei turni in senso orario e antiorario;
- Applicazione delle penalità, meccaniche per dichiarare o contestare **UNO** e gestione della **Challenge** per la carta *Wild Draw Four* (+4);
- Calcolo dei punteggi a fine round in base alle carte rimaste nelle mani degli avversari.

1.2 Struttura generale dell'applicazione

L'applicazione è organizzata in due package principali:

1. **backend**: contiene il modello dei dati (carte, colori, valori e mazzo), la gerarchia dei giocatori e i metodi di supporto alla logica di gioco;
2. **gui**: contiene l'interfaccia grafica Swing e la classe principale **FinestraPrincipale**, che coordina lo stato della partita, i turni, le regole e l'aggiornamento della grafica.

1.3 Funzionalità implementate

Il programma include le seguenti funzionalità:

- **Configurazione iniziale**: una finestra di dialogo all'avvio per impostare numero di giocatori, nomi, ruoli (giocatore umano, *Bot Casuale* o *Bot Aggressivo*), modalità di gioco e varianti attive.
- **Due modalità di partita**:

- *Partita Singola*: composta da un unico round in cui vince il primo giocatore che esaurisce le carte in mano;
- *Partita a Punti*: strutturata su più round con soglia di vittoria personalizzabile (predefinita a 500 punti), con classifica e riepilogo a fine round.
- **Bot**: include un bot con comportamento casuale (**BotCasuale**) e un bot che dà priorità alle carte azione e ai jolly (**BotAggressivo**).
- **Regole alternative**: possibilità di abilitare varianti opzionali come lo *Stacking* (accumulo di +2 e +4), il *Number Rush* (scarto di più carte con lo stesso valore numerico) e il *Seven-Zero* (rotazione delle mani con lo 0 e scambio delle mani con il 7).

Il progetto non richiede librerie esterne. Per il corretto caricamento delle immagini e del font è necessario mantenere la struttura delle cartelle prevista dal progetto e avviare l'applicazione dalla cartella radice.

2 Modalità di gioco e gestione dei punteggi

Il sistema permette di scegliere tra due modalità di gioco principali.

2.1 Partita Singola

La *Partita Singola* si articola in un **singolo round**:

- **Obiettivo**: vince il primo giocatore che riesce a scartare tutte le carte in proprio possesso.
- **Conclusione**: al momento della vittoria, la partita termina e viene mostrato un messaggio nel riquadro dei log.

2.2 Partita a Punti

La *Partita a Punti* si sviluppa su **più round consecutivi**:

- **Soglia di vittoria**: il valore predefinito è **500 punti**, configurabile all'inizio.
- **Assegnazione punti a fine round**: al termine di ogni round, il vincitore ottiene un punteggio pari alla somma dei punti delle carte rimaste in mano agli avversari.
- **Schermata di riepilogo**: al termine del round si apre la finestra dei punteggi, implementata dalla classe **SchermataPunteggi**. La finestra mostra la classifica generale aggiornata.
- **Proseguimento o vittoria**:
 - Se nessun partecipante ha raggiunto la soglia, chiudendo la schermata con il pulsante *"Inizia Prossimo Round"* viene avviato il round successivo con un nuovo mazzo e nuove mani.
 - Se un giocatore raggiunge o supera la soglia, la partita si conclude proclamando il vincitore finale.

2.3 Tabella dei punteggi e calcolo

Il conteggio dei punti al termine di ciascun round utilizza i valori standard delle carte UNO:

Tabella 1: Valore in punti delle carte di UNO.

Tipologia Carta	Valori / Simboli	Punti
Carte Numeriche	Da ZERO a NOVE (0 – 9)	Valore nominale (0 – 9 pt)
Carte Azione	SKIP, REVERSE, DRAW_TWO (+2)	20 pt
Carte Jolly	WILD, WILD_DRAW_FOUR (+4)	50 pt

Nel codice, la classe `Giocatore` espone il metodo `calcolaPuntiMano()`, che calcola la somma dei punti iterando sulle carte della mano senza modificarne il contenuto.

3 Scelte implementative e gestione delle regole

3.1 Gestione della carta Reverse a 2 giocatori

Nelle partite a due giocatori, la carta `REVERSE` viene gestita come un *Salta Turno*: chi la gioca scavalca l'avversario e mantiene di fatto il turno.

Nelle partite con più giocatori, la gestione del verso inverte il segno della variabile `direzione` (`direzione *= -1`), modificando l'avanzamento dei turni calcolato tramite aritmetica modulare:

```
indiceProssimo = (indiceCorrente + direzione + N) % N
```

Con due soli partecipanti ($N = 2$), la sola inversione del segno non basterebbe, dato che in modulo 2 muoversi di $+1$ o -1 porta sempre all'altro giocatore. Per gestire questo caso, quando $N = 2$ il metodo `applicaEffettiRegole()` esegue una chiamata aggiuntiva a `passaAlProssimoGiocatore()`: sommata al normale avanzamento di fine turno, questa operazione riporta il turno al giocatore che ha usato il `REVERSE`.

3.2 Terminazione con carta speciale come ultima carta

Quando una carta lascia il giocatore senza carte in mano, gli effetti vengono interrotti all'inizio del metodo `applicaEffettiRegole()`. Il controllo utilizzato è:

```
if (corrente.getNumeroCarte() == 0) return;
```

Nell'implementazione scelta viene quindi data priorità alla conclusione del round: gli effetti successivi della carta che chiude la mano non vengono applicati. Si tratta di una semplificazione adottata per gestire in modo diretto la terminazione della partita o del round.

3.3 Dichiarazione e contestazione di UNO

Chi rimane con una sola carta in mano deve dichiarare *"UNO!"*. La gestione include:

1. **Dichiarazione preventiva:**

- Quando il giocatore ha esattamente **2 carte in mano** e si appresta a giocarne una, può premere il pulsante *"DICHARA UNO!"*;
- Il sistema registra l'avvenuta dichiarazione tramite il campo booleano dedicato:

`haDichiaratoUnoSuccesso = true`

- Se il giocatore pesca una carta giocabile e dopo la pesca rimane con 2 carte, il sistema non la gioca automaticamente, lasciando il tempo di effettuare la dichiarazione prima dello scarto.

2. Contestazione di UNO:

- Se un giocatore rimane con 1 sola carta senza aver dichiarato preventivamente UNO, può essere contestato;
- In caso di contestazione valida, il giocatore riceve **2 carte di penalità** dal mazzo;
- I bot possono contestare automaticamente in base alla propria logica probabilistica.

3.4 Gestione della Challenge sul Wild Draw Four (+4)

La carta +4 è considerata regolare solo se il giocatore non possiede altre carte del colore che era attivo prima del jolly. Quando la variante *Stacking* è disattivata, il giocatore bersaglio può accettare la penalità oppure lanciare una **Challenge**. Nel caso di un bot, la decisione viene presa automaticamente tramite il metodo `effettuaChallenge()`.

1. Se non viene lanciata la Challenge, il bersaglio pesca **4 carte** e salta il turno.
2. Se viene lanciata la Challenge:
 - Se la giocata era **irregolare** e chi ha giocato il +4 possedeva una carta del colore precedente, l'attaccante pesca **4 carte**;
 - Se la giocata era **regolare**, il bersaglio che ha contestato pesca **6 carte** e salta il turno.

3.5 Passaggio del turno e Stacking

- Il pulsante *"SALTA"* permette al giocatore umano di terminare il turno solo dopo aver pescato almeno una carta dal mazzo; in caso contrario l'azione viene rifiutata con un messaggio.
- Durante una catena attiva di penalità nella variante *Stacking*, il passaggio del turno non è consentito: il giocatore deve rispondere con un'altra carta cumulabile (+2 o +4) oppure pescare l'intero ammontare accumulato.

4 Organizzazione del software

Il progetto separa il modello di base del gioco dai componenti grafici, pur mantenendo nella classe `FinestraPrincipale` anche il coordinamento di una parte importante della logica della partita:

- **Package backend:** contiene le classi che rappresentano carte, valori, colori, mazzo e giocatori, oltre ai controlli e ai comportamenti condivisi tra i partecipanti.
- **Package gui:** contiene i pannelli Swing, le finestre di dialogo e gli elementi grafici utilizzati per visualizzare e comandare la partita.
- **FinestraPrincipale:** è la finestra principale e il coordinatore del gioco. Riceve gli eventi dell'interfaccia, mantiene lo stato della partita, gestisce i turni e applica le regole e le varianti.

Questa organizzazione richiama la separazione tra modello, vista e controllo, senza adottare una suddivisione MVC completamente rigida.

5 Diagramma UML delle classi

5.1 Struttura e relazioni del diagramma

In Figura 1 sono illustrate le classi principali del sistema e le loro relazioni:

- **Gerarchia dei giocatori:** *Giocatore* è la classe astratta base, specializzata da *GiocatoreUmano*, *BotCasuale* e *BotAggressivo*.
- **Modello delle carte:** la classe *Carta* possiede due attributi, di tipo *Colore* e *Valore*. Ciascun giocatore gestisce la propria mano tramite una *List<Carta>*.
- **Coordinamento della GUI:** *FinestraPrincipale* collega i pannelli del tavolo e richiama le finestre di dialogo per configurazione, scelte e punteggi.

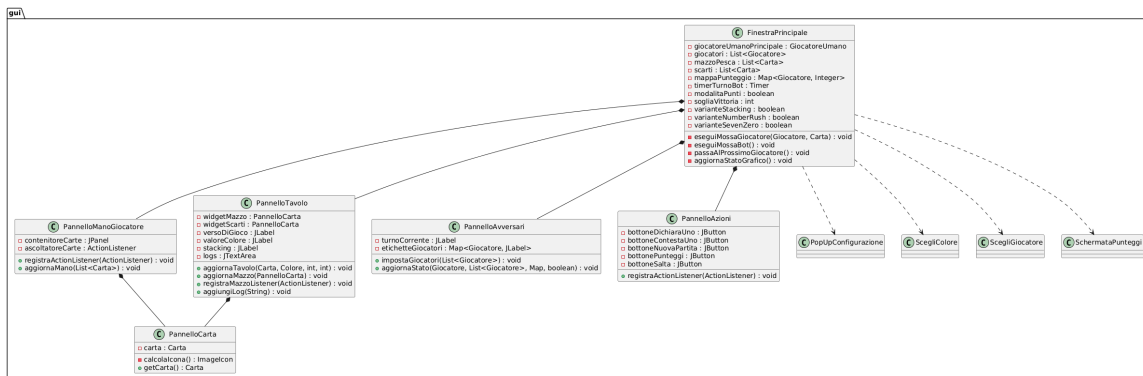


Figura 1: Diagramma UML delle classi principali.

6 Descrizione delle responsabilità delle classi principali

6.1 Package backend

- **Carta, Colore, Valore:** definiscono i componenti base delle carte e i relativi valori in punti.
- **Mazzo:** si occupa della generazione delle 108 carte e del loro mescolamento.

- **Giocatore:** classe base astratta per i partecipanti. Gestisce la mano, i controlli sulle mosse e il calcolo dei punti delle carte rimaste, e definisce le operazioni che vengono specializzate dai diversi tipi di giocatore.
- **GiocatoreUmano:** rappresenta il giocatore umano, delegando le scelte alle azioni dell'interfaccia grafica.
- **BotCasuale:** bot semplice che seleziona la prima carta valida disponibile ed effettua alcune decisioni con probabilità del 50%.
- **BotAggressivo:** bot che privilegia carte azione e jolly, sceglie il colore più frequente nella mano, seleziona per lo scambio del 7 l'avversario con meno carte ed effettua alcune decisioni con probabilità del 70%.

6.2 Package gui

- **FinestraPrincipale:** classe principale (`JFrame`) che gestisce la partita, i turni e il collegamento tra stato del gioco e interfaccia grafica.
- **PannelloTavolo:** area centrale con mazzo, scarti, indicatore colore/verso, accumulo dello Stacking e riquadro dei log di gioco.
- **PannelloManoGiocatore:** pannello inferiore con barra a scorrimento per visualizzare e selezionare le carte in mano.
- **PannelloAvversari:** pannello superiore con riassunto visivo del turno, del numero di carte e, nella modalità a punti, dei punteggi.
- **PannelloAzioni:** colonna laterale con i pulsanti di comando (Dichiara/Contesta UNO, Salta, Nuova Partita, Punteggi).
- **PannelloCarta:** componente grafico basato su `JButton` che associa una carta alla relativa immagine.
- **PopUpConfigurazione:** dialog modale per impostare partecipanti, modalità e varianti prima dell'avvio.
- **ScegliColore:** dialog utilizzato dal giocatore umano per scegliere il colore dopo un jolly.
- **ScegliGiocatore:** dialog utilizzato per scegliere il giocatore con cui scambiare la mano nella variante Seven-Zero.
- **SchermataPunteggi:** dialog con tabella ordinata per la visualizzazione della classifica dei punteggi.
- **StileUI:** classe di utilità per il caricamento del font *Fredoka*.

7 Costrutti del linguaggio Java impiegati

Nello sviluppo sono stati impiegati diversi costrutti della programmazione ad oggetti e delle API Java.

7.1 Interfacce ed eventi

- **ActionListener**: utilizzata nella GUI per la gestione degli eventi generati dai pulsanti, dalle carte e dal timer dei turni dei bot.

7.2 Generics e Collezioni

- **List<Carta>** (in particolare **ArrayList**): utilizzata per la gestione dinamica del mazzo di pesca, degli scarti e delle mani dei giocatori.
- **Map<Giocatore, Integer>**: utilizzata per memorizzare i punteggi progressivi dei partecipanti durante le partite a punti.

7.3 Stream API ed Espressioni Lambda

- **Espressioni Lambda**: utilizzate per definire in modo compatto alcuni listener Swing, il timer dei turni dei bot e altre operazioni sulle collezioni.
- **Stream API**: impiegate, ad esempio, per filtrare le carte, cercare una mossa valida, verificare la presenza di carte con determinate caratteristiche e costruire liste derivate.
- **Comparator per la classifica**: in **SchermataPunteggi** viene utilizzato lo stream ordinato con `.sorted((giocatore1, giocatore2) -> ...)` per generare la classifica in ordine decrescente di punteggio.

8 Descrizione dell'interfaccia grafica con Screenshot

8.1 Schermata di configurazione iniziale

All'avvio compare la finestra di dialogo illustrata in Figura 2. Tramite questa schermata è possibile impostare:

- La modalità di gioco (*Partita Singola* o *Partita a Punti* con soglia regolabile);
- L'attivazione delle varianti opzionali (*Stacking*, *Number Rush*, *Seven-Zero*);
- Il numero di partecipanti, da 2 a 6, con la possibilità di assegnare a ciascuno un nome personalizzato e il relativo tipo (*Umano*, *Bot Casuale* o *Bot Aggressivo*).

Configura la Partita

Modalità di gioco

☒ Partita Singola
 ☐ Partita a Punti
 Soglia vittoria: 500

Varianti di Gioco Attive

☐ Stacking
 ☐ NumberRush
 ☐ SevenZero

Numero giocatori

2 3 4 5 6

Elenco Giocatori

Giocatore 1:

Giocatore 2:

Figura 2: Schermata per la configurazione della partita.

8.2 Tavolo da gioco principale

Durante la partita (Figura 3), l'interfaccia organizza lo spazio in quattro sezioni principali:

- **Area superiore (PannelloAvversari):** mostra il giocatore di turno e il numero di carte, e nella modalità a punti mostra anche il punteggio dei partecipanti;
- **Area centrale (PannelloTavolo):** ospita il mazzo di pesca cliccabile, gli scarti, l'indicatore del colore attivo, il verso di gioco, l'eventuale accumulo dello Stacking e il riquadro scorrevole con i log degli eventi;
- **Colonna laterale (PannelloAzioni):** raggruppa i pulsanti per dichiarare o contestare UNO, visualizzare i punteggi, avviare una nuova partita e passare il turno con il pulsante *SALTA*;
- **Area inferiore (PannelloManoGiocatore):** barra a scorrimento orizzontale che visualizza le carte in mano al giocatore umano.



Figura 3: Schermata del tavolo da gioco.

8.3 Finestre di dialogo per i punteggi e le scelte

Nella modalità a punti, al termine di ciascun round viene visualizzata la finestra dei punteggi, implementata da *SchermataPunteggi*. La stessa schermata può essere aperta

durante la partita tramite il pulsante dedicato. La tabella mostra i partecipanti in ordine decrescente in base ai punti totali.



Posizione	Giocatore	Punti Totali
1°	Giocatore 1	0 punti
2°	Giocatore 2	0 punti

Inizia Prossimo Round

Figura 4: Schermata dei punteggi e della classifica.

9 Descrizione delle Regole Alternative

Nel menu di configurazione iniziale è possibile attivare tre varianti di gioco opzionali, utilizzabili singolarmente o in combinazione.

9.1 Stacking (Accumulo +2 e +4)

Quando un partecipante subisce una carta penalità (+2 o +4), non è costretto a pescare immediatamente se possiede a sua volta una carta +2 o +4 giocabile: può rispondere giocandola e aumentare l'accumulo di penalità. La catena continua finché un partecipante non risponde con un'altra carta di penalità; in quel caso deve pescare l'intero totale accumulato, visualizzato sul tavolo da gioco.

9.2 Number Rush

La variante *Number Rush* permette di scartare più carte dello stesso valore numerico nello stesso turno quando il valore della carta scelta coincide con quello della carta in cima agli scarti. Se il giocatore possiede almeno due carte con quel valore, il programma scarta tutte le carte della mano che hanno lo stesso valore numerico. Il colore dell'ultima carta scartata diventa il nuovo colore attivo sul tavolo.

9.3 Seven-Zero

Introduce effetti speciali legati alle carte numeriche ZERO e SETTE:

- **Regola dello 0:** quando viene scartato uno ZERO, tutti i partecipanti ruotano la propria mano in base al verso di gioco corrente.
- **Regola del 7:** quando viene scartato un SETTE, il giocatore che lo ha giocato scambia l'intera mano con un altro partecipante. Il giocatore umano sceglie l'avversario tramite la finestra *ScegliGiocatore*, mentre i bot utilizzano la propria strategia.

All'inizio di ogni round la prima carta degli scarti viene scelta tra le sole carte numeriche. Se viene estratta una carta azione o un jolly, questa viene reinserita nel mazzo e si procede con una nuova estrazione. Gli effetti Seven-Zero non vengono quindi applicati come effetto iniziale del round.

10 Strategie dei Bot

I bot estendono la classe astratta `Giocatore` e utilizzano gli stessi controlli di giocabilità delle carte, ma adottano criteri diversi per le decisioni automatiche.

10.1 Bot Casuale

Il `BotCasuale` implementa un comportamento semplice:

- **Selezione della mossa:** gioca la prima carta valida individuata nella propria mano;
- **Scelta del colore:** sceglie casualmente uno dei colori presenti nella propria mano; se non sono presenti carte colorate, sceglie un colore casuale tra i quattro disponibili;
- **Scambio del 7:** sceglie casualmente un avversario con cui effettuare lo scambio;
- **Dichiarazioni e sfide:** la dichiarazione di UNO, la contestazione e la Challenge sul +4 restituiscono un esito casuale con probabilità del 50%.

10.2 Bot Aggressivo

Il `BotAggressivo` adotta criteri più orientati all'uso delle carte speciali:

- **Priorità alle carte azione e ai jolly:** tra le carte giocabili dà precedenza alla prima carta appartenente al gruppo SKIP, REVERSE, DRAW_TWO, WILD, WILD_DRAW_FOUR; se non ne trova, gioca la prima carta valida;
- **Scelta del colore:** conta le carte dei quattro colori presenti nella propria mano e sceglie quello più frequente;
- **Scambio del 7:** sceglie l'avversario con il minor numero di carte;
- **Dichiarazione e contestazione di UNO:** restituisce una decisione positiva con probabilità del 70%;
- **Challenge sul +4:** se il bot ha meno di 3 carte non lancia la Challenge; se ha almeno 3 carte e l'avversario ne ha meno di 3, la lancia sempre; negli altri casi la lancia con probabilità del 70%.

Tabella 2: Confronto decisionale tra Bot Casuale e Bot Aggressivo.

Decisione	Bot Casuale	Bot Aggressivo
Criterio di scarto	Prima carta valida	Priorità a carte azione e jolly
Scelta del colore (Jolly)	Colore casuale	Colore più frequente in mano
Scambio carte (Regola 7)	Avversario casuale	Avversario con meno carte
Dichiarazione / Contesta	Probabilità 50%	Probabilità 70%
Challenge su +4	Probabilità 50%	Strategia su numero di carte

11 Istruzioni di compilazione ed esecuzione

11.1 Requisiti di sistema

- **Java:** JDK 25, coerente con l'ambiente utilizzato per il progetto;
- **Struttura cartelle:** è necessario mantenere i percorsi relativi previsti per la cartella `gui/images/` e per il font `gui/Fredoka.ttf`;
- **Interfaccia grafica:** la finestra principale è impostata a 1280×720 pixel e richiede un ambiente desktop grafico.

11.2 Compilazione ed esecuzione da riga di comando

Dalla cartella radice del progetto:

- **Compilazione:**
`javac -encoding UTF-8 -d bin backend/*.java gui/*.java`
- **Avvio della GUI:**
`java -cp bin gui.FinestraPrincipale`

Se la cartella `bin` non è già presente, è sufficiente crearla prima della compilazione.

11.3 Esecuzione tramite IDE

1. Importare la cartella del progetto `unoProject` nell'IDE prescelto;
2. Selezionare il JDK 25;
3. Impostare come *Working Directory* la cartella radice del progetto;
4. Eseguire come applicazione Java la classe `gui.FinestraPrincipale`.